



Advanced Python programming using AI tools

Lecture 12: Code Review and Git

Lecturer: Dr. Havana Rika

Today

1. Code Review
2. Working With Git

What are the Problems in a Generated Code?

Bugs

Attacks

Security Holes

Inconsistent Code

Data Leaks

What are the Problems in a Generated Code?

No Code Reuse

**Viruses From Importing
Unknown Libraries**

Poor Documentation

Unreadable Code

Poor Design

איך ה-AI הורסת את הקוד וכיצד אפשר לתקן את זה?

נדמה שכלי AI לכתיבת קוד מאיצים את קצב הפיתוח, אבל בפועל, לא רק שהם מאטים אותו, אלא גם יוצרים בעיות חמורות בטווח הארוך • מאמר זה סוקר את שלוש הבעיות המרכזיות ומציע איך לפתור אותן

ללי בר-אילן | 14:36 13/11/2025

בעיה מס' 1: מפעל לייצור כפילויות

למודלים אין זיכרון טוב. בכל פעם שמבקשים מהם לייצר כפתור או טופס, הם יוצרים אחד חדש במקום להשתמש בקוד קיים. כך, בתוך שבוע נוצרים 20 כפתורים כמעט זהים, מפוזרים בקוד – כל אחד עם הבאגים הייחודיים שלו.

בעיה מס' 2: ה-AI מחפשת מתחת לפנס

לכלי AI לפיתוח קוד יש שתי בעיות קריטיות שקשורות לקונטקסט: הראשונה היא שרוב המידע שנמצא בחלון הקונטקסט שלהם – כלומר, המידע שנגיש להם כהקשר רחב יותר לשאילתה מסוימת – פשוט לא ללוונטי. הבעיה השנייה היא שבדרך כלל, הקונטקסט שנגיש להם הוא פרויקט התוכנה שעליו המתכנת עובד באותו הרגע, בעוד שלא פעם, הפונקציונליות הדרושה נמצאת בכלל בפרויקט אחר.

בעיה מס' 3: קוד שנשבר בעולם האמיתי

כלי ה-AI משדרים ביטחון מלא בקוד שהם מייצרים, גם בבאגים. ואם זה לא מספיק, הם נוטים להתעלם ממקרי קצה או מבעיות אבטחה פוטנציאליות. התוצאה היא שהקוד יכול לפעול בצורה מושלמת בסביבה מבודדת, אך כשמשלבים אותו במערכת אמיתית ומעלים אותו לפרודקשן מתגלות כל הבעיות.

<https://www.pc.co.il/editorial/white-papers/440400/>

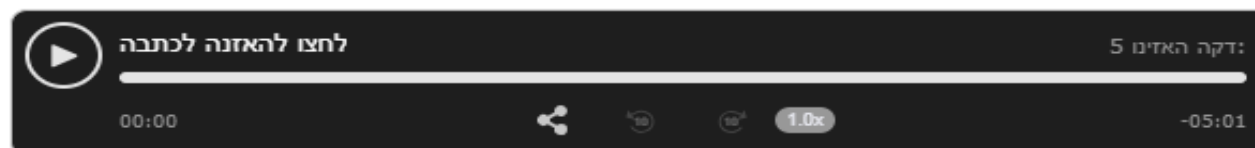


TechTalk

סכנה לעולם הפיתוח: הקוד נכתב במהירות שיא – אבל כל היתר נשאר מאחור

כלכליסט
דיגיטל

10:07, 03.08.25 אמיר איש שלום

Powered by [Trinity Audio](#)

העולם כולו מתרשם ובצדק מהיכולת של כלי AI להאיץ את כתיבת הקוד, כשבתוך שנה עברנו לכתיבת קטעי קוד שלמים בשניות. לאחרונה, מחקר של Stack Overflow מצא כי למעלה מ-70% מהמפתחים משלבים כלים מבוססי AI בעבודתם. מדובר בקפיצה טכנולוגית מרשימה, אך כבכל קפיצה מהירה מדי מישהו נשאר מאחור, ובמקרה הזה, כמעט הכל.

ה־Software Development Life Cycle (מחזור חיי פיתוח תוכנה) השתנה מקצה לקצה בכל הקשור לשלב הכתיבה. קוד שנדרש לו בעבר שבוע פיתוח, נכתב כעת בשעות. אבל שאר שלבי הפיתוח, כמו בדיקות איכות, תהליכי debugging לניפוי שגיאות, CI/CD, monitoring ועוד, נשארו כמעט ללא שינוי. זה יוצר פער מסוכן: הקוד מגיע מהר יותר לפרודקשן, אבל הארגון לא מספיק להבין אם הוא בכלל מוכן לקפיצה הזאת.

<https://www.calcalist.co.il/calcalistech/article/hjlaft3del>

קפטן אינטרנט

כששירות ממשלתי נשען על וייב קודינג: פרטים אישיים של כ-6,000 אזרחים נחשפו ברשת

מערך הדיגיטל הלאומי, שנותן מענה לגופים וליחידים בארגון של מידע, נעזר החל מפרוץ המלחמה במתנדבים לבניית תשתיות דיגיטליות ■ התוצאה היא מודל בעייתי שבנוי על תשתית לא מספקת ■ פרטים מלאים של 6,098 אזרחים שדיווחו על נזקים, כולל תמונות ומספרי תביעות במס רכוש, היו זמינים ברשת לכל אדם עם ידע טכני מינימלי

שמירה בטול זן

3



<https://www.themarker.com/captain-internet/2025-11-06/ty-article/.premium/0000019a-5355-d5d0-a59f-737f18910000>

Why We Review

- **Knowledge Sharing:** Prevents "silos" so multiple people understand the system.
- **Discover bugs earlier:** Rather than discovering bugs after a feature has been shipped and scrambling to release a patch.
- **Enhance security:** Code reviews create a high level of security, especially when security professionals engage in a targeted review.
- **Mentoring:** A primary way for junior developers to learn from seniors and for seniors to get fresh perspectives from juniors.
- **Increase collaboration:** When team members work together to create a solution, they feel more ownership of their work and a stronger sense of belonging.
- **Maintainability:** Ensuring the code is readable for the "next person" (which is usually you in 6 months).
- **Improve code quality:** Code reviews are an important way to ensure you ship high-quality code and quality software.

The Reviewer's Checklist

1. **Design & Architecture:** Does it fit the system? Is it over-engineered? Solve the problem you have today, not the one you might have in a year.
2. **Functionality:** Does it do what it's supposed to? Think about edge cases and concurrency (deadlocks/race conditions).
3. **Complexity:** Can this be understood quickly by a human? If it's too complex to explain, it's too complex to merge.
4. **Tests:** Are there unit/integration tests? Do the tests actually fail when the code is broken?
5. **Naming & Comments:** Use descriptive names. Comments should explain Why (the reasoning), while the code should explain What (the logic).
6. **Style:** Follow the official style guide. Use automation for formatting so humans don't have to discuss "tabs vs. spaces."

Code Review Using ChatGPT

list of things to **check in a code review**:

- **Syntax** errors or **bugs**
- Code **readability** (clear variable names, proper indentation)
- Missing **comments** or documentation
- Following coding **standards** (like PEP 8 for Python)
- Potential **performance** issues
- **Security** risks, like hardcoded passwords

Code Review Using ChatGPT

Prompt 1: General Code Review

A broad prompt to get an overview:

Review the following Python code. Point out any syntax errors, readability issues, missing comments, or ways to improve it. Suggest changes with examples.

Prompt 2: PEP 8 Compliance

Ensure my code followed PEP 8 (Python's style guide). This caught issues like missing spaces after commas, unnecessary parentheses and style guidelines.

Check if this Python code follows PEP 8 standards. List any violations and suggest fixes.

Prompt 3: Performance and Security

To catch bigger issues

Analyze this Python code for performance issues or security risks. Suggest improvements with examples.

Code Review Using ChatGPT

Tips for Using ChatGPT in Code Reviews

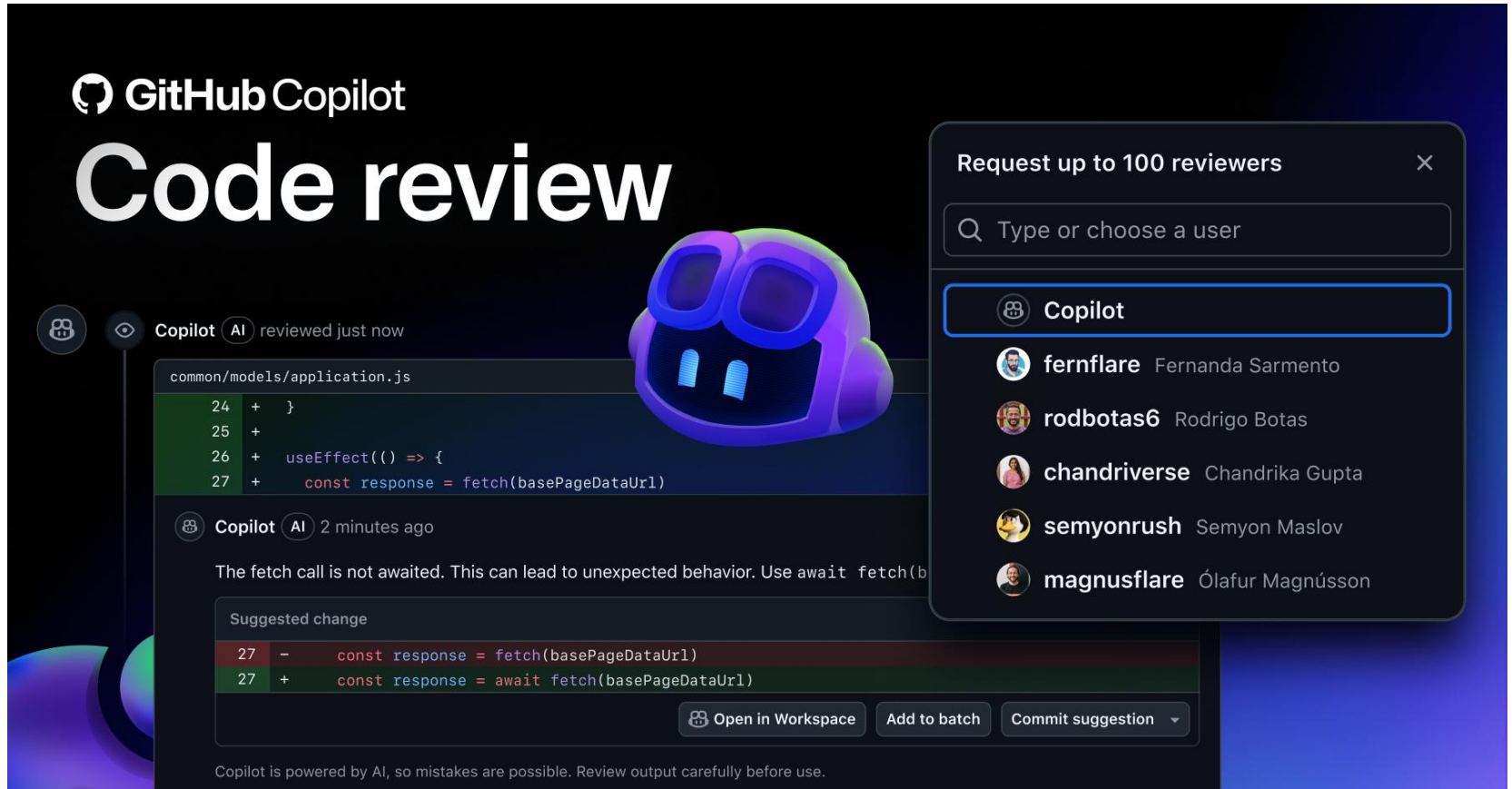
Be Specific: Write clear prompts with exactly what you want (e.g., “check for PEP 8” or “find security risks”).

Break It Down: For big files, review functions or classes one at a time to avoid overwhelming ChatGPT.

Verify Suggestions: Test any changes ChatGPT suggests to ensure they work.

Combine with Human Review: Use ChatGPT for initial checks, but do a final pass yourself for logic and design.

Code Review Using GitHub Copilot



GitHub Copilot

Code review

 **Copilot** AI reviewed just now

```
common/models/application.js
24 + }
25 +
26 + useEffect(() => {
27 +   const response = fetch(basePageDataUrl)
```

 **Copilot** AI 2 minutes ago

The fetch call is not awaited. This can lead to unexpected behavior. Use `await fetch(b`

Suggested change

```
27 -   const response = fetch(basePageDataUrl)
27 +   const response = await fetch(basePageDataUrl)
```

 Open in Workspace Add to batch Commit suggestion ▾

Copilot is powered by AI, so mistakes are possible. Review output carefully before use.

Request up to 100 reviewers ✕

🔍 Type or choose a user

-  **Copilot**
-  **fernflare** Fernanda Sarmento
-  **rodbotas6** Rodrigo Botas
-  **chandriverse** Chandrika Gupta
-  **semyonrush** Semyon Maslov
-  **magnusflare** Ólafur Magnússon

<https://www.youtube.com/watch?v=LsQGilvXAfE>

Code Review Using GitHub Copilot

Core Functionality:

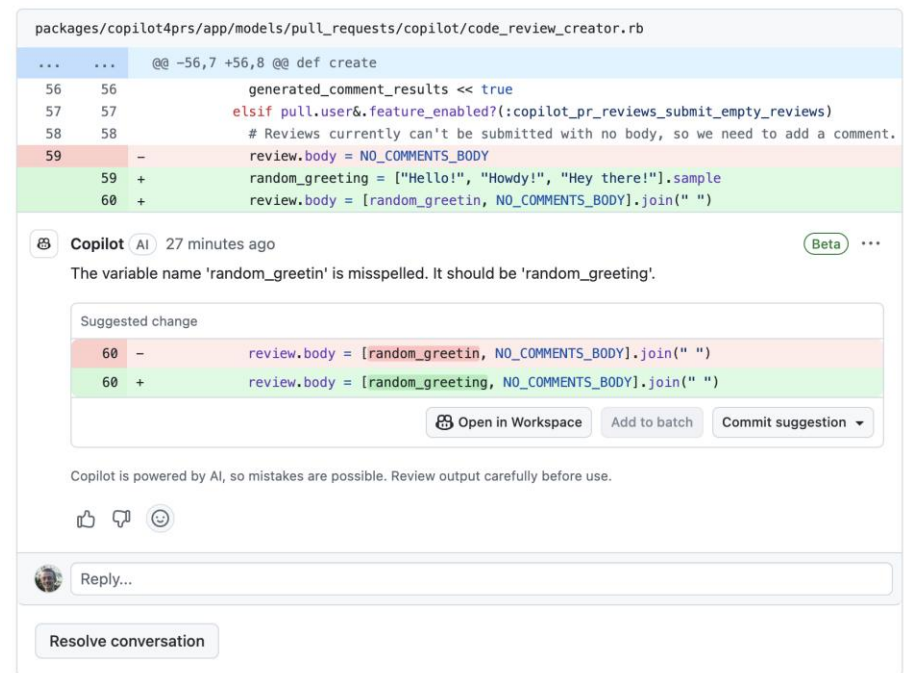
GitHub Copilot can review your local changes and provide comments.

Key Features:

- **Quick Feedback:** Reviews typically take less than 30 seconds.

- **Actionable Suggestions:** Copilot often includes specific code blocks that you can apply or implement with a single click.

- **Custom Instructions:** You can tailor the review by adding a `.github/copilot-instructions.md` file to your repository. This can include rules like "respond in Spanish," "focus on readability," or "apply specific security checklists."



<https://docs.github.com/en/copilot/how-tos/use-copilot-agents/request-a-code-review/use-code-review>

Git and GitHub Introduction

What is Git?

Git is a popular **version control** system, created in 2005.

It is used for:

- Tracking **code changes**
- Tracking **who** made **changes**
- Coding **collaboration**



What is GitHub?

GitHub makes **tools** that use **Git**.

GitHub is the **largest host** of source **code** in the world,
and has been **owned** by **Microsoft** since **2018**.

Git Operations

Repository: A folder where Git tracks your project and its history.

Clone: Make a copy of a remote repository on your computer.

Stage: Tell Git which changes you want to save next.

Commit: Save a snapshot of your staged changes.

Branch: Work on different versions or features at the same time.

Merge: Combine changes from different branches.

Pull: Get the latest changes from a remote repository.

Push: Send your changes to a remote repository.

Start Working with Git

- Initialize Git on a folder, making it a **Repository**
- Git now creates a hidden folder to keep track of changes in that folder
- When a file is changed, added or deleted, it is considered **modified**
- You select the modified files you want to **Stage**
- The **Staged** files are **Committed**, which prompts Git to store a **permanent** snapshot of the files
- Git allows you to see the full history of every commit.
- You can revert back to any previous commit.
- Git does not store a separate copy of every file in every commit, but keeps track of changes made in each commit!

Use GitHub in PyCharm



[How To Use GitHub For Beginners](#)

[PyCharm and Git Integration](#)



[How to efficiently use GitHub integration in PyCharm](#)